

Network Protocols

TCP, UDP, SSH, HTTP/HTTPS, and the Protocols That Run the Internet

A complete reference covering the layered network model, transport protocols, and the most important application-layer protocols in use today — with ports, use cases, and how they relate to each other.

Table of Contents

1. Introduction
2. The Layered Network Model
3. IP — Internet Protocol
4. TCP — Transmission Control Protocol
5. UDP — User Datagram Protocol
6. TCP vs. UDP: When to Use Which
7. HTTP — Hypertext Transfer Protocol
8. HTTPS and TLS/SSL
9. SSH — Secure Shell
10. DNS — Domain Name System
11. FTP, SFTP, and File Transfer
12. Email Protocols: SMTP, IMAP, POP3
13. Other Notable Protocols
14. Well-Known Ports Reference
15. Quick Comparison Table

1. Introduction

A network protocol is an agreed-upon set of rules that lets devices exchange data reliably, regardless of hardware or operating system. The internet is built from many protocols stacked in layers, each responsible for a specific job — from moving raw bits over a wire to rendering a web page in a browser.

This guide walks through the core transport protocols (TCP, UDP), the layered model they fit into, and the most widely used application-layer protocols: HTTP, HTTPS, SSH, DNS, FTP/SFTP, and email protocols.

2. The Layered Network Model

Networking is commonly explained using two related models: the 7-layer OSI model (theoretical) and the 4-layer TCP/IP model (practical, what the internet actually uses).

TCP/IP Layer	OSI Equivalent	Examples
Application	Application, Presentation, Session	HTTP, HTTPS, SSH, FTP, DNS, SMTP
Transport	Transport	TCP, UDP
Internet	Network	IP, ICMP
Link	Data Link, Physical	Ethernet, Wi-Fi (802.11)

Data flows down the stack when sending (application → link) and up the stack when receiving (link → application). Each layer adds its own header — this is called encapsulation.

3. IP — Internet Protocol

IP is responsible for addressing and routing packets between devices across networks. It is connectionless and makes no delivery guarantees — that reliability is added by the transport layer above it (TCP) when needed.

- **IPv4** — 32-bit addresses, e.g. 192.168.1.1. About 4.3 billion possible addresses, now largely exhausted.
- **IPv6** — 128-bit addresses, e.g. 2001:db8::1. Designed to replace IPv4 with a vastly larger address space.
- **ICMP** — used for diagnostics and error reporting (e.g. the ping and traceroute tools) rather than carrying application data.

4. TCP — Transmission Control Protocol

TCP is a connection-oriented transport protocol that guarantees reliable, ordered, error-checked delivery of a stream of bytes between two hosts.

- **Three-way handshake** — SYN, SYN-ACK, ACK establishes a connection before any data is sent.
- **Reliability** — lost packets are detected and retransmitted; data is reassembled in the correct order.
- **Flow and congestion control** — adjusts transmission rate to avoid overwhelming the receiver or the network.
- **Connection teardown** — a four-step FIN/ACK exchange closes the connection cleanly.

Trade-off: reliability and ordering add overhead and latency compared to UDP, since TCP must confirm delivery and can wait to retransmit lost data.

5. UDP — User Datagram Protocol

UDP is a connectionless transport protocol that sends independent packets (datagrams) with minimal overhead and no delivery guarantees — packets may arrive out of order, duplicated, or not at all.

- **No handshake** — a sender can transmit immediately.
- **No retransmission** — lost datagrams are simply gone unless the application layer handles that itself.
- **Lower latency and overhead than TCP**, which is why it is preferred for real-time applications.
- **Common uses:** DNS queries, video/voice calls (VoIP), online gaming, live streaming.

6. TCP vs. UDP: When to Use Which

Aspect	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery, in order	No guarantees
Speed	Slower (more overhead)	Faster (minimal overhead)
Use cases	Web, email, file transfer, SSH	DNS, VoIP, gaming, streaming
Header size	20+ bytes	8 bytes

Rule of thumb: choose TCP when correctness matters more than speed (a webpage must load completely and correctly); choose UDP when speed and low latency matter more than occasional data loss (a dropped video frame is less costly than a delayed one).

7. HTTP — Hypertext Transfer Protocol

HTTP is an application-layer protocol for transferring web content. It is a request-response protocol: a client sends a request, and a server returns a response. HTTP runs over TCP (port 80 by default) and is stateless — each request is independent unless state is added via cookies, sessions, or tokens.

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: curl/8.0

HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256

<html>...</html>
```

- **Methods** — GET (retrieve), POST (submit data), PUT (replace), PATCH (partial update), DELETE (remove), HEAD, OPTIONS.
- **Status codes** — 2xx success, 3xx redirection, 4xx client error (404 Not Found), 5xx server error (500 Internal Server Error).
- **Versions** — HTTP/1.1 (persistent connections), HTTP/2 (multiplexed streams, header compression), HTTP/3 (runs over QUIC/UDP instead of TCP for reduced latency).

8. HTTPS and TLS/SSL

HTTPS is HTTP layered on top of TLS (Transport Layer Security, the successor to SSL). It runs on port 443 by default and provides three guarantees that plain HTTP lacks:

- **Encryption** — data in transit cannot be read by eavesdroppers.
- **Integrity** — data cannot be modified in transit without detection.
- **Authentication** — the server's identity is verified via a digital certificate issued by a trusted Certificate Authority (CA).

TLS Handshake (simplified)

1. Client Hello - client proposes TLS version, cipher suites
2. Server Hello - server picks a cipher suite, sends certificate
3. Key Exchange - client and server derive a shared session key
4. Finished - both sides confirm; encrypted data can now flow

SSL is deprecated — all modern connections should use TLS 1.2 or 1.3. "SSL certificate" is still common informal terminology for what is technically a TLS certificate.

9. SSH — Secure Shell

SSH is an application-layer protocol for securely accessing and managing remote machines over an untrusted network. It runs over TCP, typically on port 22, and encrypts the entire session, including authentication credentials.

- **Remote shell access** — log into a remote machine's command line.
- **Command execution** — run a single command on a remote host without an interactive session.
- **File transfer** — SCP and SFTP both run over an SSH connection.
- **Port forwarding / tunneling** — securely tunnel other traffic (e.g. a database connection) through an encrypted SSH channel.

```
# Connect to a remote host
ssh user@example.com

# Run a single remote command
ssh user@example.com "df -h"

# Copy a file using SCP over SSH
scp local_file.txt user@example.com:/remote/path/

# Local port forwarding: forward local port 8080 to remote:80
ssh -L 8080:localhost:80 user@example.com
```

SSH authenticates using passwords or, more securely, public/private key pairs — the private key stays on the client and is never transmitted.

10. DNS — Domain Name System

DNS translates human-readable domain names (example.com) into IP addresses that computers use to route traffic. It typically runs over UDP port 53 for standard queries, falling back to TCP for larger responses (such as zone transfers).

- **A record** — maps a domain name to an IPv4 address.
- **AAAA record** — maps a domain name to an IPv6 address.
- **CNAME record** — aliases one domain name to another.
- **MX record** — specifies mail servers responsible for a domain.
- **NS record** — delegates a domain to authoritative name servers.

Resolution flow (simplified): a resolver queries a root server, which refers it to a TLD server (e.g. for .com), which refers it to the domain's authoritative name server, which returns the answer.

11. FTP, SFTP, and File Transfer

- **FTP** (File Transfer Protocol) — transfers files over TCP, traditionally on ports 20 and 21. Sends credentials and data unencrypted by default; largely considered insecure today.
- **FTPS** — FTP secured with TLS encryption.
- **SFTP** (SSH File Transfer Protocol) — an entirely different protocol from FTP that runs over SSH (port 22), providing encrypted file transfer with the same authentication as SSH.

Despite the similar name, SFTP is not "FTP over SSL" — it is a distinct protocol built on the SSH connection, and is generally the recommended modern choice.

12. Email Protocols: SMTP, IMAP, POP3

- **SMTP** (Simple Mail Transfer Protocol) — sends outgoing email between mail servers. Port 25 (server-to-server), or 587 with authentication (client submission).
- **IMAP** (Internet Message Access Protocol) — retrieves email while keeping it synchronized on the server (port 143, or 993 over TLS). Supports multiple devices viewing the same mailbox.
- **POP3** (Post Office Protocol v3) — downloads email to a single device, traditionally removing it from the server (port 110, or 995 over TLS).

13. Other Notable Protocols

- **DHCP** — automatically assigns IP addresses and network configuration to devices joining a network.
- **ARP** — resolves an IP address to a physical (MAC) address on a local network.
- **NTP** — synchronizes clocks between computers over a network.
- **WebSocket** — provides a persistent, full-duplex connection over a single TCP connection, commonly used for real-time web apps.
- **QUIC** — a modern transport protocol built on UDP that underlies HTTP/3, combining speed with built-in encryption.
- **SNMP** — used to monitor and manage network devices such as routers and switches.
- **Telnet** — a legacy remote-login protocol; unencrypted and largely replaced by SSH.

14. Well-Known Ports Reference

Port	Protocol	Transport
20 / 21	FTP (data / control)	TCP
22	SSH / SFTP	TCP
23	Telnet	TCP
25	SMTP	TCP
53	DNS	UDP / TCP
67 / 68	DHCP	UDP
80	HTTP	TCP
110	POP3	TCP
123	NTP	UDP
143	IMAP	TCP
443	HTTPS	TCP
587	SMTP (submission)	TCP
993	IMAPS (IMAP over TLS)	TCP
995	POP3S (POP3 over TLS)	TCP

15. Quick Comparison Table

Protocol	Layer	Transport	Primary Purpose
IP	Internet	—	Addressing and routing packets
TCP	Transport	—	Reliable, ordered byte streams
UDP	Transport	—	Fast, connectionless datagrams
HTTP	Application	TCP	Transfer web content
HTTPS	Application	TCP (+TLS)	Encrypted web content
SSH	Application	TCP	Secure remote access / tunneling
DNS	Application	UDP / TCP	Resolve domain names to IPs
FTP	Application	TCP	Unencrypted file transfer
SFTP	Application	TCP (via SSH)	Encrypted file transfer
SMTP	Application	TCP	Send outgoing email
IMAP	Application	TCP	Sync email across devices
DHCP	Application	UDP	Assign IP configuration
WebSocket	Application	TCP	Persistent bidirectional web link

End of guide.